Homework 2: Test Generation and Improvement Using Coding Agents

Andres Cabeza · AI Research Assistant (HW1 app) · November 2026

Repository: https://github.com/andres9403/ai-research-assistant · final suite in `backend/tests_hw2/`

Summary

A coding agent generated a pytest suite for the backend of my HW1 app, then improved it over two rounds that each used a different kind of feedback. I measured every version the same way: line and branch coverage, a manual correctness review, and how many seeded faults it detects. 40 faults were shown to the agent in Round 2; 18 more were held out and never shown.

	Initial	Round 1	Final (Round 2)
Tests run (functions)	352 (225)	379 (248)	439 (253)
Result	348 passed, 4 xfailed	375 passed, 4 xfailed	433 passed, 6 xfailed
Line / branch coverage	96.7% / 93.6%	99.8% / 97.4%	99.9% / 97.6%
Seeded faults detected (40, shown in Round 2)	37 (92.5%)	38 (95%)	**40 (100%)**
Held-out faults detected (18, never shown)	18 (100%)	18 (100%)	18 (100%)
Real application bugs found	4	4	**5**
Tests asserting only a status code	17	6	5
References to private helpers	13	15	2

The tests found five real bugs in the app (Section 2.3). In every version they run in under 8 seconds and pass on repeated runs.

1. Test Generation Approach

1.1 Setup

- **Agent.** Claude Code (Claude Opus 5.5). I used it in two roles:
  - **Generator.** A fresh sub-agent wrote and improved every test. Each round continued the same agent conversation, the way a developer keeps talking to one session.
  - **Orchestrator.** The main Claude Code session, which I directed, wrote the prompts, ran the measurements, built the seeded-fault harness and reviewed the tests. The generator never saw the harness or its code.
- **Isolation.** The repository already had a 279-test suite (`backend/tests/`) from HW1. To get a clean experiment, the agent wrote the new suite in `backend/tests_hw2/` and was told not to open the old one. The HW1 suite is kept only as a reference point.
- **Scope.** Backend only (FastAPI + SQLite, 1,960 statements in `backend/app/`), since that is where the logic lives. The frontend is out of scope.

1.2 Initial prompt

The prompt was deliberately a realistic "first ask". It gave the location, the goals (main functionality, edge cases, error handling, meaningful assertions), and the constraints:

- no reading `backend/tests/`;
- no changes to app code;
- mock all network and LLM calls;
- use a temporary data directory.

It also said: *"If a test fails and you believe the APPLICATION is wrong … mark it with `@pytest.mark.xfail(strict=True, reason="SUSPECTED BUG: ...")`"*, so that app bugs would be visible rather than "fixed" by weakening a test. It said nothing about coverage, mutation testing or specific requirements. The full prompt is in `docs/hw2/AGENT_LOG.md`.

1.3 What was generated

The agent read `CLAUDE.md`, `docs/PLANNING.md` and the source, then built its own fixtures:

- a temporary DB and data directory set before any app import;
- a per-test `.env`;
- network blocking on both `httpx` and `httpx2`, the HTTP library the LLM SDKs use;
- a fake LLM and a deterministic fake embedder;
- a `helpers.py` that builds real PDFs with PyMuPDF.

File	Tests	Targets
`test_library_service.py`	29	title/DOI normalisation, duplicate detection, save/edit/list
`test_papers_api.py`	31	`/api/papers` CRUD, 404/409/422, PDF serving, chunks
`test_search.py`	47	Semantic Scholar + arXiv parsing, fallback, filters, `/api/search`
`test_pdf_processing.py`	37	PDF validation, sections, headings, chunking, token estimate
`test_metadata.py`	32	heuristic and LLM metadata, S2/arXiv lookups
`test_upload_and_pipeline.py`	51	upload/attach, open-access download, download guard, restart recovery
`test_ai_api.py`	26	summary and Q&A endpoints, caching, citations, 409/502/503
`test_summaries_chat_retrieval.py`	37	context selection, citation renumbering, history, retrieval
`test_llm.py`	38	provider choice, Ollama/Anthropic/OpenAI response and error handling
`test_settings_api.py`	24	API-key endpoints, `.env` writing rules, key validation
Total	**352**	225 test functions; the rest are parametrised cases

By type, the suite mixes:

- unit tests of pure functions (normalisation, chunking, citation parsing, request builders);
- service tests against a real SQLite database;
- API tests through FastAPI's `TestClient`, with mocked HTTP and LLMs.

2. Initial Test Evaluation

2.1 Execution and coverage

`348 passed, 4 xfailed in 3.4 s`. I ran the suite three times with identical results, and every file also passes on its own.

Metric	Initial	HW1 reference
Line coverage	96.7% (64 statements missed)	96.6%
Branch coverage	93.6% (28 partial branches)	91.7%
Runtime	3.4 s	33 s

Important code that was not covered:

File	Coverage	Missed
`services/embeddings.py`	70%	the real `FastEmbedder`
`services/http.py`	88%	DNS failure and fallback-address paths of the guard against downloads from private addresses
`services/llm.py`	94%	the OpenAI SDK error mapping
`services/pipeline.py`	92%	rollback when writing an upload fails; embedding-failure fallbacks
`db.py`	91%	`add_missing_columns`, the schema upgrade for older databases

2.2 Test correctness

I reviewed the tests by hand and with an AST scan:

- **Every test has assertions.** None was a smoke test that only checks the app runs. Expected values were generally right and specific: exact titles, exact citation numbers, exact `.env` contents.
- **A misleading test.** `test_attach_pdf_processes_it_and_fills_only_blank_fields` claims to check the requirement "PDF metadata only fills blank fields", but every field it checks was blank. It never shows a non-blank field surviving. Its title assertion is vacuous, because the code never touches the title. So the "never overwrite" requirement had no test (seeded fault M33 survived).
- **No boundary values.** The library fixtures used years 2018 and 2021, and the filters used 2019, so whether `year_to` is inclusive was never checked (M08 survived). The title-similarity test compared a near-identical title with a completely different one, never a near miss (M38 survived).
- **Weak error-path assertions.** 17 tests asserted only a status code. Many were fine 404/422 checks. Others missed the point of the requirement, for example:

```
def test_question_on_paper_without_text_is_409(client, session, fake_llm):   # initial
    paper = add_paper(session, status="failed")
    response = client.post(f"/api/papers/{paper.id}/chat", json={"question": "Hi?"})
    assert response.status_code == 409          # ...but was the LLM called? was anything
saved?
```

- **Brittleness.** 13 references to private helpers (`llm._parse`, `metadata._clean_title`, `client._transport`, …) and about 157 exact-message assertions. These are correct today but tie the tests to the implementation.
- **No flaky tests.** The suite uses no sleeps, real clocks or randomness.

2.3 Fault detection: real bugs found

The initial suite exposed **four real application bugs**, each as a strict xfail. I checked each one against the code and the UI before accepting it. A fifth bug was mentioned in the agent's report but not tested; I confirmed

it by hand and the agent added a test for it in Round 2.

1. DOI normalisation. `normalize_doi("doi: 10.1234/ABC")` returns `" 10.1234/abc"` with a leading space: whitespace is stripped *before* the `doi:` prefix is removed, so duplicate detection misses the common printed form.
Detected by `test_library_service.py::test_normalize_doi_handles_space_after_doi_prefix`

2. Whitespace-only title. `POST /api/papers` with title `" "` returns 201, although editing refuses blank titles ("a paper always keeps a title").
Detected by `test_papers_api.py::test_save_paper_rejects_whitespace_only_title`

3. Body text dropped. In `split_sections`, the branch that relabels text before a late first heading as "Full text" is dead code: front matter always starts on page 1, so `page_start > 2` is never true. Pages of body text are left out of chunking, so summaries and Q&A never see them.
Detected by `test_pdf_processing.py::test_text_before_a_late_first_heading_is_kept_as_full_text`

4. Failed retry breaks a ready paper. `POST /pdf/fetch` is allowed on an already-processed paper; if that download fails, the status becomes `no_pdf` although the PDF and chunks are intact, so summary and Q&A return 409. Reachable through the API only: the UI hides the button when a PDF exists.
Detected by `test_upload_and_pipeline.py::test_failed_retry_keeps_an_already_processed_paper_ready`

5. Model names mangled in titles *(found in Round 2).* The footnote-mark regex strips digits after letters, so a heuristic title "YOLOv3: An Incremental Improvement" becomes "YOLOv: …" and "ResNet50" becomes "ResNet".
Detected by `test_upload_and_pipeline.py::test_upload_keeps_model_names_in_a_heuristic_title` (2 cases)

Following the assignment, the app was **not** changed to make these tests pass. They stay as strict xfails, so each one turns into a failure the moment its bug is fixed and reminds whoever fixes it to remove the marker. The HW1 suite found none of these bugs.

2.4 Fault detection: seeded faults

The real-bug count depends on what bugs happen to exist, so I also measured fault detection with **seeded faults**. Each is a one-line change that breaks a requirement stated in `PLANNING.md` or a docstring, for example:

- the guard against downloads from private addresses accepts private IPs (M31);
- the API-key format allows newlines, which would let a key inject lines into `.env` (M23);
- citations aren't renumbered (M16);
- the token budget is ignored (M20, M21).

The tool is `backend/hw2_eval/run_mutants.py`. It applies each fault to a temporary copy of the app and runs the suite there; a fault is detected if any test fails.

The 40 main faults were written before any HW2 test existed. The initial suite detected **37 of 40**, against 35 for the HW1 suite. The three it missed were M08, M33 and M38, for the reasons given in Section 2.2.

3. Iterative Test Improvement

Each round used a different strategy rather than asking for "more tests". The prompts are quoted in full in `docs/hw2/AGENT_LOG.md`.

Round 1: execution feedback and an assertion review

Context given.

1. The `--cov-report=term-missing` output, with a one-line note on what each uncovered range does. For example:

```
app/services/pipeline.py 92% missing 87–88, 137–139 (upload rolls back when writing
the PDF raises OSError), 191, 213–218 (embed(): EmbeddingError and unexpected errors
are swallowed/rolled back so the paper stays usable)
```

2. The review findings from Section 2.2: the misnamed attach test and the status-only error-path tests, each with the side effect it should check.

The prompt ended: *"Goal: stronger, behaviour-focused assertions and coverage of the important missing paths — not test count. Don't add assertions that only exist to execute lines."* The mutation results were held back.

Changes the agent made.

- **Split the misleading test.** It became `test_attach_fills_blank_fields_from_the_pdf` and `test_attach_never_overwrites_existing_fields`. The second gives the paper an existing title, authors, year, abstract and DOI, and asserts that all of them survive.
- **Error paths now assert no side effects.** No chat message or summary is saved, the LLM is never called, `.env` is not written, and an existing PDF and its chunks are untouched:

```
def test_question_on_paper_without_text_is_409(client, session, fake_llm):   # Round 1
    paper = add_paper(session, status="failed")
    response = client.post(f"/api/papers/{paper.id}/chat", json={"question": "Hi?"})
    assert response.status_code == 409
    assert "full text isn't available" in response.json()["detail"]
    assert fake_llm.calls == []
    assert session.query(ChatMessage).count() == 0
```

- **New `test_app_infrastructure.py`.**
  - It builds an old-schema `chat_messages` table by hand, starts the app, and checks that the migration adds the missing columns, that the old row still reads through the API, and that a second run changes nothing.
  - It covers serving the built frontend.
  - It tests `FastEmbedder` with a stand-in `fastembed` module, so nothing is downloaded.
- **New tests for the uncovered paths** listed in the coverage report: the download guard's DNS failure and fallback, upload rollback on `OSError`, the embedding-failure fallbacks, and the OpenAI error mapping.
- **Two of its own tests fixed.** While working, the agent found that two of its earlier tests passed for the wrong reason:
  - an Ollama "by name" test was really hitting the generic "No LLM is set up" path;
  - a chunk-merge test's input never triggered a merge.

Outcome.

Metric	Before Round 1	After Round 1

Line coverage	96.7%	99.8%
Branch coverage	93.6%	97.4%
Status-only tests	17	6
Seeded faults detected	37	38

The one extra fault detected was M33, caught by the corrected attach test. **The roughly 3-point coverage gain caught no further seeded faults.** Executing the remaining lines didn't reach M08 or M38, because those need different *input values*, not different *lines*.

Round 2: mutation feedback, bug triage and a requirements audit

Holding out faults first. Before Round 2, I wrote **18 more seeded faults (H01–H18) that the agent never saw**. Round 2 tells the agent which main faults survived, so its score on those 40 could be inflated by targeting them. The held-out set guards against that.

Context given.

- **(A) Mutation report.** The two surviving faults, in the style a mutation tool prints them, with an instruction not to target them one by one:

```
app/services/library.py, list_papers():  `stmt.where(Paper.year <= year_to)`  →  `... < year_to`
app/services/metadata.py:  `TITLE_MATCH_THRESHOLD = 0.9`  →  `TITLE_MATCH_THRESHOLD = 0.5`
Don't just write a test aimed at each exact mutation: work out what weakness in the suite let each survive … and fix that class of weakness across the suite.
```

- **(B) Bug triage.** My verdict on each suspected bug: bugs 1–3 confirmed; bug 4 confirmed but reachable through the API only; bug 5 confirmed, to be tested through the public upload API rather than the private helper.
- **(C) Requirements audit.** Check every user-visible requirement in `PLANNING.md` against the suite.
- **(D) Maintainability.** Replace private-helper calls with public routes, and remove redundant tests.

Changes the agent made.

- **(A) Boundary tests across the suite.** The agent named the blind spot itself: *"my tests only used inputs well inside or well outside a limit, never the value at the limit"*. It then added at-the-limit tests everywhere a limit exists:
  - year ranges, including both bounds and single-year ranges;
  - title similarity at 0.903 (accepted) and 0.889 (rejected) against the 0.9 threshold;
  - length limits for query (300/301), author, venue, title (7/8/300/301) and keys (19/20/400);
  - the three 50 MB size caps;
  - chunk target and minimum sizes;
  - heading length and font-size cut-offs;
  - the retrieval, summary and history token budgets.

  To check its own work it seeded 24 extra boundary faults in a scratch copy. That run exposed two more of its tests that were weak; one asserted `1 + retrieval.TOP_K`, so the expected value moved together with the mutation. It fixed both.

- **(B) Bug 5 test.** A strict xfail that uploads a real PDF titled "YOLOv3: An Incremental Improvement" with no LLM and checks the stored title.
- **(C) 13 untested requirements now tested.** Among them:
  - the library, chat and summary survive a restart;
  - a summary sends ≤ 8k tokens of the paper, and Q&A ≤ 3k tokens of excerpts ("never whole papers");
  - API keys are never logged;
  - a saved key takes effect without a restart;
  - Ollama is listed last.

  It also listed what can't be tested here: real providers, retrieval quality of the real embedding model, the UI, and the start scripts.
- **(D) Maintainability.**
  - Every private-helper call in a test was replaced by a public route; the 2 references left are the same `llm._build.cache_clear()` cache reset, at setup and teardown of one fixture, which has no public alternative. For example, the download guard is now tested end to end through the real `httpx` transport, with fake DNS and sockets, instead of by inspecting `client._transport`.
  - Ligature handling is now tested through `extract()` on a real PDF with ligature glyphs.
  - 24 redundant tests were deleted or merged.

Outcome. `433 passed, 6 xfailed`. The suite now detects **40/40 main faults and still 18/18 held-out faults**, and has 2 private references left.

4. Final Evaluation

4.1 Comparison

Measure	HW1 reference	Initial	Round 1	Final
Test functions / tests run	202 / 279	225 / 352	248 / 379	253 / 439
Lines of test code	3,353	3,059	3,467	3,721
Assertions per test function	3.13	2.62	2.79	2.88
Line coverage	96.6%	96.7%	99.8%	**99.9%**
Branch coverage	91.7%	93.6%	97.4%	**97.6%**
Main seeded faults (40)	35	37	38	**40**
Held-out seeded faults (18)	16	18	18	**18**
Real bugs exposed	0	4	4	**5**
Status-only tests	8	17	6	5
Private-helper references	1	13	15	**2**
Runtime	33 s	3.4 s	3.0 s	3.6 s

Final coverage. 2 statements are still uncovered:

- `pdf.py:304` is the dead branch behind bug 3, so it cannot run until that bug is fixed;
- `pdf.py:167` is a whitespace-only text line that a generated PDF can't produce.

The 11 partial branches left are loop exits.

Final correctness. The final suite contains no test known to be incorrect:

- the 4 tests that passed for the wrong reason were all fixed by the agent;
- every xfail reason was checked against the code;
- the 5 remaining status-only tests are input-validation checks (404/422), where the status code is the behaviour under test.

4.2 What worked and what didn't

Strategy	Effect
A human review of what assertions miss (side effects, misleading names)	**Effective.** Directly caught M33 and removed most weak tests.
The coverage report as feedback	**Mixed.** It raised coverage by 3 points and found worthwhile untested code (migration, error mapping, rollback), but caught no further seeded faults. High coverage was not the same as strong tests.
Mutation results framed as a class of weakness	**Most effective per word.** Two surviving faults led the agent to a general diagnosis (no at-the-limit inputs) and about 30 boundary tests, and the held-out set confirmed no regression.
Confirming or rejecting suspected bugs	**Effective, and needs a human.** The agent can't decide intent by itself; for example, bug 4 is only reachable through the API, which needed the UI code to judge.
A requirements audit against `PLANNING.md`	**Effective.** It found 13 untested requirements; the token-budget ones are central to the app's design.
Asking to reduce coupling and redundancy	**Effective.** Private references fell from 15 to 2, and 24 tests were removed with no loss of fault detection.

Caveat on the seeded-fault numbers. The final 40/40 on the main set is partly the result of telling the agent which faults survived. The held-out set was meant to check that, but every version detected all 18, so it hit a ceiling: it shows Round 2 didn't over-fit at the cost of other faults, but it can't show further improvement. The fairest evidence that Round 2 strengthened the suite generally is:

- the boundary tests the agent added beyond the two faults it was told about;
- the agent's own 24 extra boundary faults;
- the 13 requirements that had no test before.

5. Reflection

How well did the agent understand the app? Very well. From `PLANNING.md`, the docstrings and the code it worked out:

- the domain: duplicate detection, citation renumbering, token budgets;
- the security intent: the guard against downloads from private addresses, and the key format that prevents `.env` injection;
- the testing conventions: temp data directory, everything mocked.

Unprompted, it found three real bugs from the gap between a docstring or comment and the code. Bug 3, a dead branch, is the kind of defect that is easy to miss when reading code.

What it did well:

- realistic fixtures, including real PDFs rather than mocks;
- exact expected values;
- error-path coverage;

- security-relevant tests.

It also respected the "don't weaken a test to make it pass" instruction: suspected bugs became strict xfails, not deleted assertions.

Where it struggled:

1. **Boundary values.** It tested comfortably inside and outside each limit, which looks thorough and gets high coverage, yet misses off-by-one faults.
2. **Tests that pass for the wrong reason.** It wrote four, which it found only when asked to re-examine its tests or check them with mutants.
3. **Coupling.** It reached for private helpers when a public route existed.
4. **Deciding whether a behaviour is a bug.** For example, "YOLOv3" being stripped: it noticed the issue but did not write a test, because it could not tell whether the behaviour was intended.

How useful were context and feedback? The type of feedback mattered more than the amount:

- Coverage numbers mostly bought coverage.
- A review, and especially mutation results, bought test strength. Two surviving mutants, framed as "find the class of weakness", produced a better change than the whole coverage report.
- Human judgement about intent (the bug triage) was the one input the agent could not produce itself.

Lessons about coding agents for testing:

- An agent can produce a large, fast, well-structured suite in minutes.
- "All tests pass" and "96% coverage" both overstate quality. Seeded faults and a critical read of the assertions are needed to see what a suite really checks.
- The most productive loop was the agent generating tests, a human or tool measuring them in a way the agent can't game, and the agent being told *why* the tests are weak, not just *where*.

6. GitHub Repository

- **Repository:** https://github.com/andres9403/ai-research-assistant
- **Final suite:** `backend/tests_hw2/` (11 test files plus `conftest.py` and `helpers.py`).
- **Seeded-fault harness:** `backend/hw2_eval/`.
- **Raw results:** `docs/hw2/results/` (pytest output, coverage JSON and mutation JSON for every version).
- **Agent prompts:** `docs/hw2/AGENT_LOG.md`.
- **Suite snapshots:** commits `666b421` (initial) and `84f5a35` (Round 1).

```
cd backend
python -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest tests_hw2 -q                                    # 433 passed, 6
xfailed
.venv/bin/python -m pytest tests_hw2 -q --cov=app --cov-branch --cov-report=term-missing
.venv/bin/python -m hw2_eval.run_mutants tests_hw2 --out /tmp/main.json            # 40 faults
.venv/bin/python -m hw2_eval.run_mutants tests_hw2 --set heldout --out /tmp/held.json  # 18
faults
```

No API keys or network access are needed; every external call is mocked. A plain `pytest` still runs only the HW1 suite (`backend/pytest.ini`). The HW2 suite runs in its own process, because each suite's conftest points the app at its own temporary database. CI runs both suites on every push. More detail, including how to score an earlier snapshot, is in `docs/hw2/README.md`.